micro-ROS on the ESP32-S3

A Practical Manual for the Autonomous Waste-Sorting Robot

Alvin Sam Omondi Gamba

October 20, 2026

Revision 1

About this manual

This manual explains how micro-ROS runs on the ESP32-S3 of this robot, how to use and modify the firmware, how the web dashboard talks to it, and what to do when something goes wrong. It is written for a student who has not used micro-ROS before. Read Chapter 1, then follow Chapter 2 step by step to build and start the system, then read Chapter 3 to understand it. Chapters 4–6 are reference. Appendix B explains the terms.

How to read the marks. A dagger † marks a statement that has not been confirmed on the robot or against the installed library version. They are collected in Appendix A. Boxed notes give the short practical rule for a topic.

Conventions. Wheels FL, FR, RL, RR are named from the robot's own point of view (standing behind it, looking forward). The body frame is x forward, y left, z up; positive yaw is counter-clockwise, the same as `angular.z` in a `Twist` message.

Contents

About this manual **1**

1 System Overview **4**

1.1 Two Computers, One Robot 4

1.2 Hardware on the ESP32-S3 4

1.3 Where Each Job Runs 4

2 Setting Up From Scratch **6**

2.1 What You Need 6

2.2 Install WSL 2 and Ubuntu 24.04 6

2.3 VS Code Connected to Ubuntu 7

2.4 Install the Basic Tools 7

2.5 USB Passthrough: Giving the ESP32 to Linux 7

2.6 Install ROS 2 Jazzy 8

2.7 Install the micro-ROS Agent 9

2.8 Install the Other Host Packages 10

2.9 Install PlatformIO 10

2.10 The Project Folder 10

2.11 platformio.ini, Line by Line 10

2.12 custom.meta: the Entity Limits 12

2.13 First Build and Flash 12

2.14 First Run: Seeing It Connect 12

2.15 Daily Operation 13

2.16 Moving to the Raspberry Pi 14

3 How micro-ROS Works **16**

3.1 The Idea 16

3.2 Three Settings That Must Match 16

3.3 Entities: What the Firmware Creates . . . 17
3.4 Entity Limits . . . 17
3.5 Execution Model: One Loop, No Threads . . . 18
3.6 Connection Recovery . . . 18
3.7 How Much Data the Serial Link Carries . . . 18
3.8 Quality of Service . . . 19
3.9 Clock Synchronisation . . . 19
3.10 Checking That It Works . . . 19

4 How the Firmware Works **21**
4.1 Robot Constants . . . 21
4.2 From Command to Wheels . . . 21
4.3 Wheel Speed Control . . . 22
4.4 The IMU . . . 22
4.5 Attitude and Position Estimate . . . 23
4.6 Heading Hold . . . 23
4.7 Self-Check . . . 23
4.8 Actuators and Failsafe . . . 24
4.9 Where to Change What . . . 24

5 Interfaces and the Web Dashboard **25**
5.1 Message Conventions . . . 25
5.2 Starting the Dashboard . . . 25
5.3 What Each Panel Does . . . 25
5.4 Xbox Controller . . . 26
5.5 Safety Behaviour . . . 26
5.6 Dashboard and Firmware Mismatches . . . 26

6 Troubleshooting **28**
6.1 Connection and Flashing . . . 28
6.2 Sensors and Actuators . . . 29
6.3 Dashboard and Network . . . 30

A Items Not Yet Verified **31**

B Glossary **32**

Chapter 1

System Overview

1.1 Two Computers, One Robot

The robot has two computers (fig. 1.1). The **ESP32-S3** does everything that needs fast, reliable timing: it spins the wheels at a commanded speed, counts encoder pulses, moves the servos and the conveyor, reads the IMU and colour sensor, estimates position, and stops the robot if commands stop arriving. A **host computer** (a laptop under WSL 2 while developing, a Raspberry Pi on the robot) runs ROS 2 Jazzy: the micro-ROS agent, rosbridge for the web dashboard, `twist_mux`, and the navigation software (lidar, SLAM, Nav2).

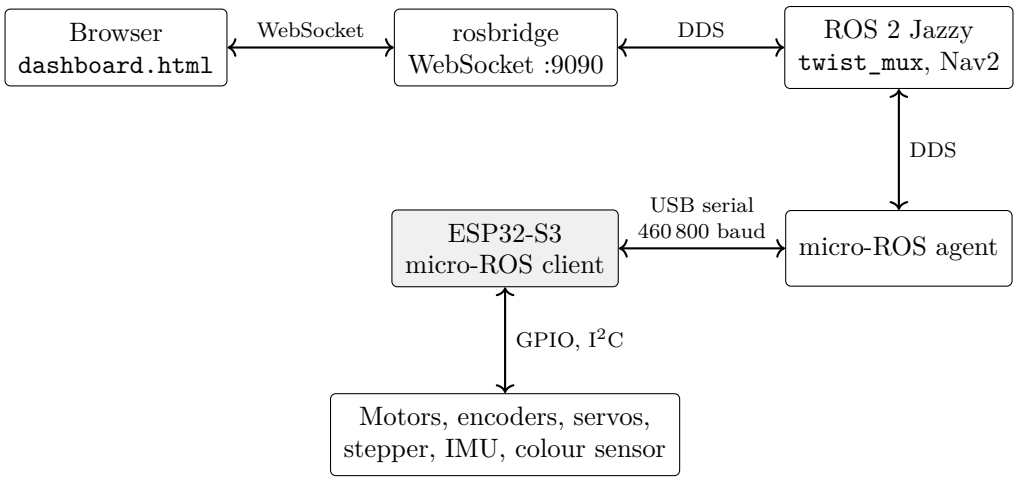


Figure 1.1: Architecture. The shaded box is what this manual concentrates on.

1.2 Hardware on the ESP32-S3

Pin cautions GPIO46 (`ENC_FL_A`) is a strapping pin on the ESP32-S3 and `ENC_RL_A` is marked "read 0 during test" in `pins.h`. If an encoder reads nothing, check these two first. `PAN_PIN` is still defined but there is no pan servo any more.

1.3 Where Each Job Runs

Table 1.1: Peripherals (pin numbers live only in `pins.h`).

Peripheral	**Interface and notes**	**Qty**
DC drive motors	H-bridge: `IN1` carries PWM, `IN2` sets direction; `MOTOR_DIR_*` flips a motor wired backwards	4
Wheel encoders	Two GPIOs each, 748 counts per wheel revolution	4
Tilt and dump servos	50 Hz hobby PWM; tilt limited to 30°–150°	2
Conveyor stepper	28BYJ-48, four coil pins, via `AccelStepper`	1
IMU	MPU6050 clone, I^2C `0x68`; `WHO_AM_I` reads `0x72`	1
Colour sensor	TCS34725, I^2C `0x29`	1

Table 1.2: Division of work.

Job	**Runs on**
Wheel speed control, encoder counting, servos, stepper, IMU and colour reads	ESP32
Attitude filter, position estimate (EKF), heading hold	ESP32
Command timeout and self-check	ESP32
Bridging serial to ROS 2 (micro-ROS agent)	Host
Choosing between manual and Nav2 commands (`twist_mux`)	Host
Dashboard web server (port 8000) and rosbridge (port 9090)	Host
Lidar, SLAM, Nav2, camera and vision	Host (Pi)

Chapter 2

Setting Up From Scratch

This chapter takes you from a clean Windows laptop to a robot that shows up in ROS 2. Follow it in order and do not skip the checks at the end of each section; each check proves that step worked before you build on it. Commands in a grey box are typed exactly as shown. Lines beginning with `PS>` run in Windows PowerShell; everything else runs in the Ubuntu terminal.

2.1 What You Need

Table 2.1: Requirements.

Item	Detail
Computer	Windows 11 laptop, virtualisation enabled in BIOS, about 20 GB free disk (the micro-ROS library and agent builds are large), internet
Board	ESP32-S3 development board (this robot: `esp32-s3-devkitc-1`, 16 MB flash, octal PSRAM) with two USB ports: one native USB, one UART bridge
Cable	A data USB cable (charge-only cables show no device)
Project files	`main.cpp`, `pins.h`, `gains.h`, `attitude.h`, `MPU6050_Custom.h/.cpp`, `custom.meta`, `platformio.ini`, `run_all.sh`, `twist_mux_config.yaml`, `dashboard.html`
Time	About 1.5–2 hours the first time, mostly downloads and builds

2.2 Install WSL 2 and Ubuntu 24.04

ROS 2 Jazzy targets Ubuntu 24.04. On Windows it runs inside WSL 2, a lightweight Linux virtual machine.

```
PS> wsl --install -d Ubuntu-24.04      # run PowerShell as Administrator
```

Restart if asked, open “Ubuntu 24.04” from the Start menu, and choose a Linux username and password (you need the password for `sudo`). Then confirm the version from PowerShell and inside Ubuntu:

```
PS> wsl -l -v                          # VERSION column must say 2
PS> wsl --update                       # newer kernel, needed for USB and
mirrored networking
```

```
lsb_release -a                          # inside Ubuntu: expect Ubuntu 24.04
```

Check: `wsl -l -v` shows `Ubuntu-24.04` with version 2. If it shows 1, run `wsl -set-version Ubuntu-24.04 2`.

2.3 VS Code Connected to Ubuntu

Work inside Ubuntu from VS Code so the editor, terminal and PlatformIO all see the same files.

1. Install VS Code on Windows and open it.
2. Extensions (`Ctrl+Shift+X`): install **WSL** (Microsoft).
3. Click the green `><` icon at the bottom left and choose **Connect to WSL**.
4. `File > Open Folder` and open your project folder (we create `~/ROBOCAR_ROS` in section 2.10).
5. `Terminal > New Terminal`: this is an Ubuntu shell.

2.4 Install the Basic Tools

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y git curl cmake build-essential python3-pip python3-venv
\
software-properties-common locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8
```

`apt update` refreshes the list of available packages; `upgrade` installs updates; ROS 2 requires a UTF-8 locale.

2.5 USB Passthrough: Giving the ESP32 to Linux

WSL 2 is a virtual machine, so a USB device plugged into Windows belongs to Windows until you hand it over with `usbipd-win`.

One-time install (Windows).

```
PS> winget install --interactive --exact dorssel.usbipd-win
```

Close and reopen PowerShell afterwards.

Which USB port? The board has two connectors. The **UART** port goes through a USB-serial chip (CP210x or CH340) and appears in Linux as `/dev/ttyUSB0`. The **native USB** port appears as `/dev/ttyACM0`. **Use the UART port.** Pressing the board's reset button on the native port drops the USB connection and Windows grabs it back (Error 1). The firmware is built to send micro-ROS data on the UART pins (`ARDUINO_USB_CDC_ON_BOOT=0`), so the UART port is also the one that is wired for it.

Share the device (administrator PowerShell).

```
PS> usbipd list                          # find the board, note its BUSID, e.
g. 1-4
PS> usbipd bind --busid 1-4              # once per device; survives reboots
PS> usbipd attach --wsl --busid 1-4      # every time the board is re-plugged
 or WSL restarts
```

Check in Ubuntu:

```
ls /dev/ttyUSB* /dev/ttyACM*             # expect /dev/ttyUSB0
```

If nothing appears, run `dmesg | tail` in Ubuntu. A message about a missing driver means the WSL kernel does not include the driver for your board's USB-serial chip†; run `wsl -update`, and if it persists, try another board revision or cable.

Permission to use the port.

```
sudo usermod -a -G dialout $USER
curl -fsSL https://raw.githubusercontent.com/platformio/platformio-core/
develop/platformio/assets/system/99-platformio-udev.rules | sudo tee /etc/
udev/rules.d/99-platformio-udev.rules
sudo udevadm control --reload-rules && sudo udevadm trigger
```

Then run `wsl -shutdown` in PowerShell, reopen Ubuntu, and re-attach the board. Without this you get "Permission denied" (`Errno 13`) when flashing or running the agent. **Check:** the command `groups` lists `dialout`.

2.6 Install ROS 2 Jazzy

```
sudo add-apt-repository universe
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.
key -o /usr/share/keyrings/ros-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/
ros-archive-keyring.gpg] http://packages.ros.org/ros2/ubuntu $(. /etc/os-
release && echo $UBUNTU_CODENAME) main" | sudo tee /etc/apt/sources.list.d/
ros2.list > /dev/null

sudo apt update
sudo apt install -y ros-jazzy-ros-base ros-dev-tools python3-colcon-common-
extensions

echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

The first block adds the ROS package source and its signing key; `ros-base` is ROS 2 without graphical tools (smaller, fine for a robot); the last two lines make every new terminal load ROS automatically. **Check:**

```
printenv ROS_DISTRO                    # jazzy
ros2 --help                            # prints the command list
```

Optional end-to-end test of ROS 2 itself: `sudo apt install ros-jazzy-demo-nodes-cpp`, then run `ros2 run demo_nodes_cpp talker` in one terminal and `ros2 run demo_nodes_cpp listener` in another.

2.7 Install the micro-ROS Agent

The agent is the host-side half of micro-ROS (chapter 3). Without it the ESP32 has nobody to talk to.

Option A: pre-built package (try first).

```
apt search micro-ros-agent
sudo apt install -y ros-jazzy-micro-ros-agent
```

A binary package named `ros-jazzy-micro-ros-agent` is expected to exist†. If `apt` cannot find it, use Option B.

Option B: build from source. This is the method used on this project. It takes a while.

```
sudo rosdep init                       # once per machine; ignore "already
exists"
rosdep update

mkdir -p ~/microros_ws/src
cd ~/microros_ws/src
git clone -b jazzy https://github.com/micro-ROS/micro_ros_setup.git

cd ~/microros_ws
rosdep install --from-paths src --ignore-src -y
colcon build
source install/local_setup.bash

ros2 run micro_ros_setup create_agent_ws.sh     # downloads the agent
sources
ros2 run micro_ros_setup build_agent.sh         # compiles them (slow)
source install/local_setup.bash
```

Make the agent available in every terminal:

```
echo "source ~/microros_ws/install/local_setup.bash" >> ~/.bashrc
```

If you built from source, also uncomment the matching `source` line in `run_all.sh`. **Check:**

```
ros2 pkg list | grep micro_ros_agent    # prints micro_ros_agent
ros2 run micro_ros_agent micro_ros_agent --help
```

2.8 Install the Other Host Packages

```
sudo apt install -y ros-jazzy-rosbridge-suite ros-jazzy-twist-mux
```

`rosbridge` lets the web dashboard talk to ROS 2 over a WebSocket. `twist_mux` chooses between manual and Nav2 velocity commands. **Check:** `ros2 pkg list | grep -E "rosbridge|twist_mux"` lists both.

2.9 Install PlatformIO

PlatformIO builds and flashes the ESP32 firmware. On Ubuntu 24.04 the `apt` version of `pio` is broken, so install it with the official installer, which puts it in `~/.platformio/penv`.

```
curl -fsSL -o get-platformio.py https://raw.githubusercontent.com/platformio
/platformio-core-installer/master/get-platformio.py
python3 get-platformio.py
echo 'export PATH=$PATH:$HOME/.platformio/penv/bin' >> ~/.bashrc
source ~/.bashrc
pio --version                          # prints PlatformIO Core, version 6.
x
```

You may also install the **PlatformIO IDE** extension in VS Code (it uses the same installation). **Check:** `pio -version` works.

2.10 The Project Folder

`run_all.sh` runs from its own folder and expects the dashboard and mux config beside it, so keep one project root:

```
~/ROBOCAR_ROS/
platformio.ini          build settings (next section)
custom.meta             micro-ROS entity limits
run_all.sh              starts agent, rosbridge, twist_mux, web server
twist_mux_config.yaml   manual vs Nav2 priorities
dashboard.html          web interface
include/                pins.h  gains.h  attitude.h  MPU6050_Custom.h
src/                    main.cpp  MPU6050_Custom.cpp
```

Create it and make the script executable:

```
mkdir -p ~/ROBOCAR_ROS/include ~/ROBOCAR_ROS/src
cd ~/ROBOCAR_ROS
chmod +x run_all.sh
```

`gains.h` is generated; never edit it by hand.

2.11 platformio.ini, Line by Line

The file below is rebuilt from the earlier manual and the current `main.cpp` includes. Compare it with your own `platformio.ini`[†]: the current one was not available when this was written. The

library line for `BasicLinearAlgebra` is new because the EKF needs it.

```
[env:esp32-s3-devkitc-1]
platform = espressif32
board = esp32-s3-devkitc-1
framework = arduino

board_upload.flash_size         = 16MB
board_build.partitions          = default_16MB.csv
board_build.arduino.memory_type = qio_opi
board_build.flash_mode          = qio
board_build.psram_type          = opi

board_microros_transport = serial
board_microros_distro    = jazzy
board_microros_user_meta = custom.meta

build_flags =
-DBOARD_HAS_PSRAM
-mfix-esp32-psram-cache-issue
-DCORE_DEBUG_LEVEL=0
-DARDUINO_USB_CDC_ON_BOOT=0

lib_deps =
https://github.com/micro-ROS/micro_ros_platformio
madhephaestus/ESP32Encoder @ ^0.11.3
madhephaestus/ESP32Servo @ ^3.0.5
waspinator/AccelStepper @ ^1.64
adafruit/Adafruit TCS34725 @ ^1.4.4
tomstewart89/BasicLinearAlgebra

upload_port = /dev/ttyUSB0
```

`platform, board, framework` Target chip family, exact board definition, and the Arduino programming framework.

`board_upload... and board_build...` Flash size and memory layout of the module. They match a 16 MB flash, 8 MB octal-PSRAM module; if your module has no PSRAM or less flash, remove or change these lines.

`board_microros_transport = serial` micro-ROS talks over the serial port (the alternative is Wi-Fi/UDP).

`board_microros_distro = jazzy` Must match the ROS 2 version on the host.

`board_microros_user_meta` Points to `custom.meta` (section 2.12).

`ARDUINO_USB_CDC_ON_BOOT=0` Keeps `Serial` on the UART pins instead of native USB (Error 5).

`CORE_DEBUG_LEVEL=0` Turns off Arduino debug output, which would corrupt the micro-ROS byte stream.

`-mfix-esp32-psram-cache-issue` Carried over from the earlier build; leave as is[†].

`lib_deps` Libraries downloaded automatically on first build.

`upload_port` The serial port used for flashing.

Do not print to Serial The firmware uses `Serial` as the micro-ROS data link. Any `Serial.print` or opening the PlatformIO serial monitor will corrupt the stream and break the connection. For debugging, publish a topic or blink an LED instead.

2.12 custom.meta: the Entity Limits

```
{
  "names": {
    "rmw_microxrcedds": {
      "cmake-args": [
      "-DRMW_UXRCE_MAX_NODES=1",
      "-DRMW_UXRCE_MAX_PUBLISHERS=6",
      "-DRMW_UXRCE_MAX_SUBSCRIPTIONS=7",
      "-DRMW_UXRCE_MAX_HISTORY=4"
      ]
    },
    "microxrcedds_client": {
      "cmake-args": [
      "-DUCLIENT_CUSTOM_TRANSPORT_MTU=1024"
      ]
    }
  }
}
```

This file is read when the micro-ROS library is compiled, not when your firmware is compiled. That is why **editing it has no effect until the library is rebuilt**: run `pio run -t clean_microros`† and build again. What the values mean is in section 3.4.

2.13 First Build and Flash

```
cd ~/ROBOCAR_ROS
~/.platformio/penv/bin/pio run          # build; the first time downloads
and compiles micro-ROS (several minutes)
~/.platformio/penv/bin/pio run -t upload   # flash over /dev/ttyUSB0
```

(`pio` alone works once the PATH line from section 2.9 is active.) The first build is slow because it builds the whole micro-ROS library for the ESP32-S3; later builds are fast. Success ends with `[SUCCESS]`.

Flashing rules.

1. **Stop the agent first.** It holds the serial port, so uploading fails (Errors 3 and 4).
2. The board must be attached to WSL (section 2.5), and `ls /dev/ttyUSB0` must work.
3. If upload cannot connect, hold the board's `BOOT` button, tap `EN`, release `BOOT`, and retry.
4. Since pin 46 is strapping pin, disconnect it otherwise it won't upload
5. After flashing, tap `EN` once so the program starts from a clean boot.

2.14 First Run: Seeing It Connect

Before you power on: the robot must be *still and level* (the IMU calibrates for about 3 s at boot, section 4.4) and, for the first test, **on blocks with wheels off the ground**.

Terminal 1: start the agent.

```
ros2 run micro_ros_agent micro_ros_agent serial --dev /dev/ttyUSB0 -b 460800
```

`serial` selects the serial transport, `-dev` the port, `-b` the baud rate (must equal `MICROROS_BAUD` in the firmware). Add `-v6` for more log detail. Tap `EN` on the board. The agent log should now report that a client connected and a session was created, followed by lines creating the participant, topics, publishers and subscribers. Leave this terminal open.

Terminal 2: look at the graph.

```
ros2 node list                          # /base_controller
ros2 topic list                         # encoders, imu_data, odometry/
filtered, ...
ros2 topic hz /odometry/filtered        # about 20 Hz
ros2 topic echo /encoders --qos-reliability best_effort --once
```

Sensor topics are best-effort, so `echo` must ask for best-effort (section 3.8); a plain `echo` may show nothing.

Step	Expected
Agent log	Session established, entities created
`ros2 node list`	`/base_controller`
`ros2 topic list`	12 topics from table 3.2, including `/self_check` and `/headi`
`ros2 topic hz /imu_data`	about 20 Hz
Spin a wheel by hand	Its `/encoders` value changes
Wheels on blocks, publish a command (below)	Wheels turn; stop 0.5 s after the last command

First-success checklist. A first drive test without the dashboard (the firmware subscribes to `cmd_vel_raw`):

```
ros2 topic pub -r 10 /cmd_vel_raw geometry_msgs/msg/Twist "{linear: {x:
0.1}}"
```

Press Ctrl+C: within about half a second the wheels stop (the failsafe). If a step fails, see chapter 6.

2.15 Daily Operation

Starting. Robot on the floor or blocks, still and level, powered. In the project folder:

```
./run_all.sh /dev/ttyUSB0 460800
```

This starts the agent, rosbridge (port 9090), `twist_mux` (configured by `twist_mux_config.yaml`, output to `/cmd_vel_raw`) and a web server (port 8000); Ctrl+C stops all of them. Open `http://localhost:8000/dashboard.html`.

Order that avoids problems.

1. Attach the board: `usbipd attach -wsl -busid X-X`.

2. Start `run_all.sh`. The ESP32 may boot before or after it; it reconnects by itself.
3. Confirm the dashboard badge is green and telemetry is live.
4. Before flashing: stop `run_all.sh`.

Phone access. WSL 2 hides its ports from the Wi-Fi by default. Use mirrored networking (steps in Error 13), then open `http://<laptop-wifi-ip>:8000/dashboard.html` on a phone on the same Wi-Fi.

2.16 Moving to the Raspberry Pi

The Pi takes the laptop's role. Nothing in the firmware changes. The Pi model is not yet fixed, so check model-specific notes†.

1. Flash **Ubuntu Server 24.04 (64-bit)** with Raspberry Pi Imager; preset hostname, user, SSH and Wi-Fi. Connect with SSH (VS Code Remote-SSH works).
2. `sudo apt update && sudo apt upgrade -y`, then check the clock with `timedatectl status`. The Pi has no battery clock, so install `chrony` (`sudo apt install chrony`) if it is wrong; a wrong clock breaks stamps (Error 19).
3. Repeat section 2.6 (identical on arm64), then install the packages the robot needs:

```
   sudo apt install -y ros-jazzy-ros-base python3-colcon-common-
extensions \
   ros-jazzy-rosbridge-suite ros-jazzy-twist-mux ros-jazzy-navigation2 \
   ros-jazzy-nav2-bringup ros-jazzy-slam-toolbox ros-jazzy-robot-
localization \
   ros-jazzy-robot-state-publisher
```

   Verify names with `apt search` if one is not found†.
4. Install the agent (section 2.7); building from source on a Pi is slow.
5. `sudo usermod -a -G dialout $USER`, then log out and in. There is no `usbipd` step: plug the ESP32 straight into the Pi.
6. Copy the project folder (section 2.10) to the Pi with `git`. Firmware is still flashed from the laptop unless PlatformIO is installed on the Pi.
7. **Stable port names.** With the ESP32 and a lidar both on USB serial, `ttyUSB0` and `ttyUSB1` can swap between boots, sending the agent to the lidar. Give each a fixed name:

```
   udevadm info -a -n /dev/ttyUSB0 | grep -E 'idVendor|idProduct|serial'
   # /etc/udev/rules.d/99-robocar.rules  (replace XXXX/YYYY with your
values)
   SUBSYSTEM=="tty", ATTRS{idVendor}=="XXXX", ATTRS{idProduct}=="XXXX",
ATTRS{serial}=="XXXX", SYMLINK+="esp32"
   SUBSYSTEM=="tty", ATTRS{idVendor}=="YYYY", ATTRS{idProduct}=="YYYY",
SYMLINK+="lidar"
   sudo udevadm control --reload-rules && sudo udevadm trigger
```

   If both use the same chip and have no serial number, match on the USB port instead (see `/dev/serial/by-path/`). Then run `./run_all.sh /dev/esp32 460800`.

8. Open the dashboard from the phone at `http://<pi-ip>:8000/dashboard.html`. If `sudo ufw status` shows an active firewall, allow TCP ports 8000 and 9090.

Chapter 3

How micro-ROS Works

3.1 The Idea

A normal ROS 2 node needs a DDS library, plenty of memory and an operating system. A microcontroller has none of these in spare. micro-ROS splits the node in two. A small *client* library on the ESP32 speaks a compact protocol (DDS-XRCE) over a serial cable. A program on the host, the *agent*, receives those messages and does the real DDS work on the ESP32's behalf. To every other ROS 2 program, the ESP32 simply looks like a node called `base_controller`.

Table 3.1: What sits between your code and the ROS 2 graph.

Layer	What it does here
Your code (`main.cpp`)	Callbacks, control loops, filling messages
`rclc`	Easy functions: create node, publisher, subscription, executor
`rcl` and `rmw_microxrcedds`	ROS client core mapped onto the micro-ROS protocol; holds fixed-size entity tables
Micro XRCE-DDS client	Session with the agent; reliable and best-effort streams
Serial transport	`set_microros_serial_transports(Serial)` at `MICROROS_BAUD`
UART bridge chip	The USB port on the board (`ARDUINO_USB_CDC_ON_BOOT=0` keeps `Serial` on it)
micro-ROS agent (host)	`micro_ros_agent serial -dev <port> -b <baud>`

3.2 Three Settings That Must Match

Most micro-ROS problems are one of three mismatches.

1. **Baud rate.** The firmware's `MICROROS_BAUD` (460 800) and the agent's `-b` argument must be equal. `run_all.sh` takes the baud as its second argument, default 460 800. A mismatch gives no error, just a silent link. A comment in `main.cpp` still mentions 921 600; it is out of date.

2. **Entity limits.** micro-ROS reserves fixed-size tables at build time. If the firmware creates more publishers or subscriptions than the tables hold, creation fails (section 3.4).

3. **QoS (reliability).** A best-effort publisher only reaches best-effort subscribers (section 3.8).

3.3 Entities: What the Firmware Creates

After the agent is found, the firmware creates the node, five publishers and seven subscriptions. Sensors are published *best-effort*, so a busy link drops a sample instead of stalling the robot. Commands are received *reliable*.

Table 3.2: All entities of node `base_controller`.

Topic	Type	Dir.	Notes
`encoders`	`std_msgs/Int32MultiArray`	pub	20 Hz; [FL, RL, FR, RR, stamp sec, stamp nsec]
`imu_data`	`sensor_msgs/Imu`	pub	20 Hz; frame `imu_link`
`odometry/filtered`	`nav_msgs/Odometry`	pub	20 Hz; `odom` → `base_link`; EKF output used by Nav2
`trash_color`	`geometry_msgs/Vector3`	pub	10 Hz; raw R, G, B in `x`, `y`, `z`
`self_check`	`std_msgs/Int32`	pub	1 Hz and after each run; bit-packed result
`cmd_vel_raw`	`geometry_msgs/Twist`	sub	m/s and rad/s; `angular.x`> 0.5 asks for heading hold
`pan_tilt_cmd`	`geometry_msgs/Point`	sub	`y` = tilt in degrees; `x` ignored
`dump_cmd`	`std_msgs/Float32`	sub	Dump servo angle, clamped to 15°–165°
`conveyor_cmd`	`std_msgs/Float32`	sub	Stepper speed in steps/s, ±700
`max_speed_cmd`	`std_msgs/Float32`	sub	Obsolete: does nothing, kept so old clients do not break
`self_check_cmd`	`std_msgs/Int32`	sub	Any value starts the self-check
`heading_hold_cmd`	`std_msgs/Bool`	sub	Master enable for heading hold

The last two subscriptions (`self_check_cmd`, `heading_hold_cmd`) are created last and are allowed to fail without stopping the robot. If `/self_check` or `/heading_hold_cmd` is missing from `ros2 topic list`, see section 3.4.

3.4 Entity Limits

The limits are set in `custom.meta`, which `platformio.ini` points to with `board_microros_user_meta`.

Table 3.3: Limits in `custom.meta` against what the firmware uses.

Parameter	Set	Used	Spare
`RMW_UXRCE_MAX_NODES`	1	1	0
`RMW_UXRCE_MAX_PUBLISHERS`	6	5	1
`RMW_UXRCE_MAX_SUBSCRIPTIONS`	7	7	0
`RMW_UXRCE_MAX_HISTORY`	4	—	—
`UCLIENT_CUSTOM_TRANSPORT_MTU`	1024	—	—

Adding a subscription? There is no spare subscription slot. Raise `RMW_UXRCE_MAX_SUBSCRIPTIONS` in `custom.meta`, force a micro-ROS library rebuild (`pio run -t clean_microros` is the expected command[†]), and raise the executor handle count in `create_entities()` to match.

The MTU of 1024 bytes is larger than the largest message (`Odometry`, about 720 bytes), so messages are not split.

3.5 Execution Model: One Loop, No Threads

The firmware is a single loop. There are no extra tasks and no locks, so the I^2C bus can only be used by one thing at a time without any protection. Each pass of `loop()` does these jobs in order: step the connection state machine and handle incoming commands; run the self-check if requested; run the wheel speed loop (every 50 ms); publish encoders (20 Hz); read the IMU, update the filters and publish IMU and odometry (20 Hz); read and publish colour (10 Hz); and apply the command timeout.

Rule Never call something slow inside `loop()`. The conveyor needs `runSpeed()` called very often; any delay shows up as a stuttering stepper.

Table 3.4: Calls that block the loop (durations from the source constants).

Call	When	Blocks up to
IMU calibration	Boot; robot must be still and level	about 3.2 s
Ping, waiting for agent	Every 500 ms	100 ms
Ping, connected	Every 500 ms	200 ms
Clock sync at connect	Each (re)connection	1 s
Clock sync, periodic	Every 5 min	200 ms
Self-check	Operator-triggered	several seconds

3.6 Connection Recovery

If the agent is restarted or the cable is pulled, the ESP32 must notice and start over; the old session is useless. The firmware does this with a four-state loop.

Table 3.5: Connection states.

State	Does	Moves on when
`WAITING_AGENT`	Pings the agent every 500 ms	A ping succeeds
`AGENT_AVAILABLE`	Creates node and entities	Success, or back to waiting on failure
`AGENT_CONNECTED`	Spins the executor; pings every 500 ms	4 pings in a row fail
`AGENT_DISCONNECTED`	Destroys entities	Immediately, then waits again

Four failed pings take about 1.7 s by calculation (the source comment says about 3 s). After reconnecting, heading hold is switched back on. While the agent is missing the robot still runs its sensors, its estimator and its failsafe; only publishing and command reception stop. This replaces the earlier `while(1)` halt, which left the board hanging if it booted before the agent (see Error 2 in chapter 6).

3.7 How Much Data the Serial Link Carries

At 8N1 each byte costs 10 bit-times, so a link carries $baud/10$ bytes per second: 11.5 kB/s at 115 200 and 46.1 kB/s at 460 800.

Table 3.6: Estimated payload per topic (calculated from the message definitions, not measured).

Topic	Bytes	Hz	B/s
`imu_data`	≈ 320	20	6 400
`odometry/filtered`	≈ 720	20	14 400
`encoders`	≈ 36	20	720
`trash_color`	≈ 24	10	240
`self_check`	≈ 4	1	4
Total			≈ 21 800

$$U_{115200} \approx \frac{21.8}{11.52} \approx 1.9, \qquad U_{460800} \approx \frac{21.8}{46.08} \approx 0.47.$$

At 115 200 baud the robot asks for almost twice what the cable can carry, which makes the session drop and restart at random. At 460 800 baud about half the link is used. If you must use a slower baud rate, raise `IMU_ODOM_PUB_DIV` (publish IMU and odometry every Nth cycle). Check the real figures with `ros2 topic bw`.

3.8 Quality of Service

DDS only connects a publisher to a subscriber if the publisher's reliability is at least what the subscriber asks for. A *reliable* subscriber never receives from a *best-effort* publisher.

- All five ESP32 publishers are best-effort. Any program that reads them (RViz, Nav2, `robot_localization`, rosbridge) must subscribe best-effort. A reliable subscriber receives nothing, silently.[†]
- The ESP32's subscriptions are reliable. A normal (reliable) publisher such as `twist_mux` connects fine.

Diagnosis `ros2 topic info /imu_data -v` lists the QoS of every endpoint. The dashboard telemetry staying empty while the drive buttons work is the typical sign of a QoS mismatch.

3.9 Clock Synchronisation

The ESP32 asks the agent for the current time at connection and every five minutes, then stamps IMU, odometry and encoder messages with host time. If synchronisation fails the stamps silently fall back to the ESP32's own clock, which Nav2 and the EKF cannot use (symptom: "extrapolation" or "message too old" errors). Treat a failed sync as a fault.

3.10 Checking That It Works

Run on the host after every firmware or agent change:

```
ros2 node list                    # expect /base_controller
ros2 topic list                   # every topic of the entities table
ros2 topic hz /odometry/filtered  # about 20 Hz
ros2 topic bw /odometry/filtered  # compare with the budget table
```

```
    ros2 topic info /imu_data -v      # BEST_EFFORT publisher
```

Chapter 4

How the Firmware Works

This chapter follows a command from `cmd_vel_raw` to the wheels, then the sensors to `odometry/filtered`. Source files: `main.cpp`, `attitude.h`, `MPU6050_Custom.h/.cpp`, `gains.h`, `pins.h`.

4.1 Robot Constants

Table 4.1: Constants at the top of `main.cpp`.

Meaning	Constant	Value
Wheel radius	`WHEEL_RADIUS`	0.0381 m
Encoder counts per wheel turn	`TICKS_PER_REV`	748
Distance per count	`M_PER_TICK`	0.32 mm
Track used for odometry	`TRACK_WIDTH`	0.215 m
Track used for turn commands	`TRACK_CMD`	0.30 m
Wheel speed at full command	`MAX_WHEEL_RPM`	120 rpm = 0.479 m/s
Maximum yaw rate accepted	`MAX_YAW_RATE`	2.0 rad/s

Two track widths exist on purpose: a skid-steer robot turns as if its track were wider than it is, because the tyres scrub sideways. To tune `TRACK_CMD`, command 1 rad/s, read `angular_velocity.z` on `imu_data`, and multiply `TRACK_CMD` by 1 divided by the measured value. A comment in the source gives the geometric track as 0.20 m, which disagrees with 0.215 m.

4.2 From Command to Wheels

Commands arrive on `cmd_vel_raw`, the output of `twist_mux`, which takes manual commands (`cmd_vel_teleop`, priority 50) over Nav2 commands (`cmd_vel_nav`, priority 10), each with a 0.5 s timeout. The firmware limits the turn rate to ± 2 rad/s, slows forward speed as turning increases, mixes left and right wheel speeds, scales both together if one exceeds the limit (keeping the curve shape), and hands each wheel a target in rpm:

$$v_L = v - \tfrac{1}{2} b_{\text{cmd}}\, \omega, \qquad v_R = v + \tfrac{1}{2} b_{\text{cmd}}\, \omega, \qquad \omega = \omega_{\text{cmd}} + \omega_{\text{hold}}.$$

The two wheels on each side share the same target.

4.3 Wheel Speed Control

Each wheel has its own controller, run every 50 ms. It computes a feed-forward guess from the target and corrects it with a PI term:

$$u = K_{ff}\, r + \mathrm{sgn}(r)\, B + K_p\,(r - n) + I, \qquad I \leftarrow I + K_i\, T\,(r - n),$$

where r is the target and n the measured wheel speed in rpm, $T = 0.05\,\mathrm{s}$, and u is the PWM value (limited to ± 250). The integrator stops growing when the output is saturated (anti-windup) and is cleared when the target changes sign or is near zero. The measured speed comes from encoder counts, so its resolution is $60/(748 \cdot 0.05) = 1.6\,\mathrm{rpm}$ per count.

The gains come from `tuned_gains.json` through `json_to_gains_h.py` into `gains.h` (do not edit `gains.h` by hand). Each wheel has separate forward and reverse gains. Wheels 1–4 in the JSON are FL, FR, RL, RR.

Table 4.2: Gains from `gains.h`.

Wheel	Dir.	K_p	K_i	K_{ff}	B
FL	fwd / rev	0.05 / 0.12	1.60 / 3.58	0.569 / 1.176	137.5 / 7.1
FR	fwd / rev	0.11 / 0.04	3.69 / 1.31	1.178 / 0.566	8.6 / 138.3
RL	fwd / rev	0.14 / 0.04	3.84 / 1.26	1.226 / 0.544	5.7 / 138.1
RR	fwd / rev	0.05 / 0.13	1.33 / 3.53	0.578 / 1.251	137.5 / 6.5

Pattern worth knowing On all four wheels, whenever the raw PWM sign is positive the offset B is about 138 and $K_{ff} \approx 0.57$; when negative, $B \approx 7$ and $K_{ff} \approx 1.2$. It follows the electrical direction, not the direction of travel. The likely cause is the PWM scheme (coast versus brake in the off-time)[†]. This is why gains are stored per direction. If you swap a motor or its wiring, re-run the calibration.

4.4 The IMU

The MPU6050 is read over $\mathrm{I^2C}$ in one 14-byte transaction, each 16-bit value built from two explicitly ordered byte reads (a one-line form with `Wire.read()<<8 | Wire.read()` can swap the bytes; that was Error 9). The filter is set to about 44 Hz to cut motor vibration, with the default ranges $\pm 2\,\mathrm{g}$ and $\pm 250^\circ/\mathrm{s}$.

Calibration happens in layers:

1. Fixed offsets typed into the driver, measured on this chip: accelerometer x -0.377, y $+0.211$, z $+0.639\,\mathrm{m/s^2}$ and gyro x -0.0454, y $+0.0148\,\mathrm{rad/s}$.
2. At boot, the gyro-z bias (about 1.2 s) and then the "level" reference (about 2 s).
3. While running, the gyro-z bias is nudged ($b \leftarrow b + 0.02\,\omega_z$) when the robot has been commanded to stand still for over a second.

Boot rule Power the robot on only when it is still and level on flat ground. Otherwise the gyro bias and the "level" reference are wrong and heading will drift faster. The standing-still update uses the *command*, not the encoders, so pushing the robot by hand spoils it.

The driver comment mentions a z-axis scale factor of 0.793, but the code does not apply one. Turn the robot through a known angle and compare with the dashboard yaw to decide if one is needed.

4.5 Attitude and Position Estimate

Roll and pitch. A complementary filter in `attitude.h` blends the gyro (accurate short-term) with the accelerometer's gravity direction (accurate long-term, time constant 1 s). When the measured acceleration differs from gravity by more than 15 %, such as during hard acceleration, the accelerometer is ignored. It also sets a `ramp` flag when pitch exceeds 6°; nothing reads that flag yet.

Position. A four-state extended Kalman filter tracks $[x, y, \psi, v]$ (position, heading, speed) at 20 Hz. Heading is the integrated gyro; speed is predicted from the pitch-compensated forward acceleration and corrected by the encoder speed; position follows from both:

$$v^- = v + a_f \Delta t, \quad \psi^- = \psi + \omega_z \Delta t, \quad x^- = x + v^- \cos\psi^- \, \Delta t, \quad y^- = y + v^- \sin\psi^- \, \Delta t.$$

In practice the filter is dead reckoning with a speed correction: the position and yaw "measurements" are copies of the prediction (and the yaw innovation is forced to zero because skid-steer encoders under-read turns), so only the speed difference $v_{enc} - v^-$ changes the state. The consistency gate on encoder yaw therefore has no effect at present†. Heading is gyro-only and will drift slowly; correct it from the lidar on the host.

Honest description Do not describe this as "fusing encoder heading". It fuses encoder *speed* with the accelerometer. The dashboard hint under the heading dial says "gyro + encoders"; that text is wrong.

4.6 Heading Hold

Motors differ, so an open-loop robot drifts sideways even with perfect heading knowledge. When driving straight with hold requested, the firmware stores the current yaw as a target and adds a correction to the turn rate:

$$\omega_{\text{hold}} = \text{clamp}\big(K_P(\psi^* - \psi) - K_D\,\omega_z,\ \pm 0.3\big), \qquad K_P = 1.5,\ K_D = 0.15.$$

Hold is active only if the sender sets `angular.x`> 0.5 (the dashboard does, Nav2 does not), the master switch is on, the IMU works, and the turn command is near zero. The gains were tuned before the command changed to rad/s, so re-tune them on the floor. Sign test: rotate the robot counter-clockwise by hand and confirm yaw increases on the dashboard.

4.7 Self-Check

The self-check runs only when asked (dashboard button), never at boot, and is ignored while a drive command is active. It blocks the loop for several seconds, so **put the robot on blocks**: the wheels spin. Results go out on `self_check` as one integer: 2 bits per item (0 not run, 1 pass, 2 fail, 3 ran) starting from item 0 in the least significant bits, plus bit 30 which flips after each completed run so a viewer can tell a new result from an old republish.

Servos and the stepper cannot report back, so they show "ran" (code 3): you must watch them. Motor codes carry extra meaning in the implementation: 1 both directions counted (pass); 2 forward leg failed; 3 forward worked but reverse failed; 0 both legs counted with the *same* sign. The test restores the encoder counts afterwards, so odometry does not jump.

Table 4.3: Item order in the `self_check` integer (firmware, 9 items).

Index	Item	Test
0	MPU6050	ID register, gravity magnitude, gyro offset
1	TCS34725	ID register only
2–5	Motors FL, RL, FR, RR	Forward then reverse; ≥ 5 encoder counts each way, opposite signs
6	Tilt servo	Moved only; no feedback
7	Dump servo	Moved only
8	Conveyor stepper	Moved only

4.8 Actuators and Failsafe

Conveyor. Driven through `AccelStepper`; `runSpeed()` runs every loop pass above 0.1 steps/s and the coils are de-energised below that. Maximum 700 steps/s.

Servos. Tilt (30°–150°) and dump (15°–165°, rest at 15°) stay attached at all times. The earlier attach/detach trick for the dump servo and the pan servo are gone. Whether the earlier servo ghosting (Error 7) returns has not been re-tested.†

Colour sensor. The latest result is read in one short burst every 100 ms without waiting; the first sample after start-up is discarded.

Command timeout. Every `cmd_vel_raw` resets a timer. After 500 ms without one, the firmware zeroes the command and stops the wheels once. This works even when the agent is lost, because it is outside the connection logic.

4.9 Where to Change What

Table 4.4: Tuning cheat sheet.

To change	Edit
A pin or motor polarity	`pins.h` (`MOTOR_DIR_*`)
Wheel speed response	Re-run calibration, regenerate `gains.h`
Turn behaviour	`TRACK_CMD`, `MAX_YAW_RATE`
Straight-line holding	`HOLD_KP`, `HOLD_KD`, `HOLD_MAX`
Serial speed	`MICROROS_BAUD` here and `-b` on the agent
Publish rate of IMU and odometry	`IMU_ODOM_PUB_DIV`
Command timeout	`CMD_TIMEOUT_MS` in `pins.h`
Number of publishers or subscriptions	`custom.meta` (section 3.4)

Chapter 5

Interfaces and the Web Dashboard

5.1 Message Conventions

Velocity commands are in real units: `linear.x` in m/s and `angular.z` in rad/s, which lets Nav2 drive the robot directly. The dashboard scales its own speed slider; the firmware no longer applies a speed limit. Encoders are published in the order **FL, RL, FR, RR**; keep this in mind when plotting. Table 3.2 lists every topic.

5.2 Starting the Dashboard

On the host (laptop or Pi), `run_all.sh` starts everything in one go:

```
./run_all.sh /dev/ttyUSB0 460800
```

It starts the micro-ROS agent, rosbridge (WebSocket on port 9090), `twist_mux` with output remapped to `/cmd_vel_raw`, and a web server on port 8000 serving the folder that contains the script. Then open `http://<host-ip>:8000/dashboard.html`. Pressing Ctrl+C stops all four. If the micro-ROS agent was built from source, uncomment the `source ~/microros_ws/install/setup.bash` line in the script.

The page connects to rosbridge on the same machine that served it. Two URL options are useful:

- `?host=<ip>` connects to rosbridge on another machine (needed if you open the file directly from disk);
- `?debug` shows outgoing commands in a corner box without any robot, for testing the page.

The top bar shows the connection badge (green when rosbridge is reachable; it retries with growing delay up to 10 s) and the controller badge.

5.3 What Each Panel Does

Drive. A speed-limit slider (0.1–1.0, scaling 0.479 m/s and 2.0 rad/s), a hold-to-drive pad, and keyboard control: W/A/S/D or arrow keys, Space to stop. Commands are resent every 150 ms (pad) or 100 ms (keyboard), and releasing sends a zero command. Every drive command sets `angular.x`=1, which asks for heading hold.

Sorting & actuators. Camera pan and tilt sliders (5° steps) with a centre button, *Dump trash* (175°) and *Reset bin* (5°), and a conveyor slider (±700 steps/s) with full forward, stop and full reverse.

Telemetry. Heading dial with yaw and gyro rate, the four encoder counts, acceleration, roll and pitch with acceleration magnitude, and a live colour swatch with raw R, G, B. A group is dimmed ("stale") until its first message arrives and again if the connection drops.

System self-check. A button that, after a confirmation, triggers the self-check and shows each item. It shows "running" until the result's toggle bit flips, and a message after 20 s with no result (usually because the robot was driving).

5.4 Xbox Controller

The dashboard supports a controller with the browser's *standard* mapping. The page must be focused. Everything is rebinding-friendly through `GP_MAP` in the script, and the line under the drive pad prints the index of the last button pressed.

Table 5.1: Default controller mapping.

Input	Action
D-pad (buttons 12–15)	Drive: forward, reverse, turn left, turn right
Right trigger / left trigger (7 / 6)	Conveyor forward / reverse (analogue)
Right stick (axes 2, 3)	Camera pan and tilt (rate controlled)
A / X / Y (0, 2, 3)	Dump / reset bin / centre camera
RB / LB (5 / 4)	Speed limit up / down by 0.1

5.5 Safety Behaviour

The page sends a stop when the tab is hidden or the window loses focus. Behind it are the mux timeout (0.5 s) and the firmware timeout (500 ms), so a dropped Wi-Fi link stops the robot within about half a second. The dashboard has *no software emergency-stop button*: the style rules for one exist but no button is on the page. Space, the Stop button or releasing a control are the stop actions, and none of them stops the conveyor. Rosbridge has no authentication: anyone on the same network can publish to the robot's topics.

5.6 Dashboard and Firmware Mismatches

These are the differences found between `dashboard.html` and the current firmware. Items 1 and 2 give wrong information to the user and should be fixed first.

Table 5.2: Known mismatches.

#	Where	Problem	Fix
1	Self-check list	Dashboard has 10 names including "Pan servo"; firmware reports 9 items. Every actuator is shown one row too low (tilt appears as "Pan", dump as "Tilt", stepper as "Dump") and "Conveyor stepper" is always "Not run"	Remove "Pan servo" from `SC_NAMES`; actuator group becomes items 6, 7, 8; initial states array of 9
2	Motor results	Codes 0 and 3 are shown as "Not run" and "Ran, check by eye" but for motors mean "same-sign counts" and "reverse failed"	Return distinct codes from the firmware or add motor-specific labels
3	Heading hint	Says "gyro + encoders"	Change to "gyro only"
4	Pan slider	No pan servo; `x` is ignored	Remove the slider and gamepad axis binding
5	Slider ranges	Tilt 0–180 against firmware clamp 30–150; dump 175/5 against clamp 165/15	Match the ranges or accept clamping
6	Speed limit	`/max_speed_cmd` is created but never published; the firmware ignores it anyway	Remove from both sides
7	Libraries	`roslib` and `eventemitter2` are loaded from a CDN, so the viewing device needs internet	Download both and serve from port 8000 for offline use
8	Emergency stop	CSS only, no button; gamepad "stop" action exists but is unbound	Add a button that stops drive and conveyor

Chapter 6

Troubleshooting

Each entry lists what you see, why it happens, and the fix. Errors 1–13 are the original log, updated for the current firmware; 14–19 were found while rewriting this manual. Commands marked with `PowerShell` run on Windows.

6.1 Connection and Flashing

Error 1: "Serial port not found. error 2, waiting for connection"

Cause: Pressing `EN/RST` while on the native USB port drops power on the USB line, and Windows takes the port back from WSL. **Fix:** Unplug and replug, then in an administrator PowerShell run `usbipd attach -wsl -busid X-X`. Use the UART bridge port (`/dev/ttyUSB0`), which stays powered during resets.

Error 2: node missing from `ros2 node list`

Cause: The ESP32 booted before the agent and the old firmware stopped in a `while(1)` loop. **Status:** Solved by the connection state machine (section 3.6); the board now waits for the agent and reconnects by itself. If the node still does not appear, check Errors 5 and 14.

Error 3: upload fails with "device reports readiness... returned no data"

Cause: The agent is holding the serial port. **Fix:** Stop the agent (Ctrl+C, or stop `run_all.sh`) before uploading.

Error 4: upload fails with `[Errno 2] No such file or directory`

Cause: The port vanished (board unplugged, or Windows took it back) or the agent is running. **Fix:** Stop the agent, run `usbipd attach -wsl -busid X-X` in PowerShell, and confirm with `ls /dev/ttyU*`.

Error 5: agent stuck at "logger setup"

Cause: `Serial` was going to the (unconnected) native USB pins. **Fix:** `-DARDUINO_USB_CDC_ON_BOOT=0` in `build_flags`, and make sure the agent baud matches the firmware (now 460 800).

Error 6: Adafruit BusIO compile error

Cause: Missing dependency while building the TCS34725 library. **Fix:** `#include <SPI.h>` at the very top of `main.cpp`.

Error 14: connects, then restarts at random ("session lost")

Cause: The serial link is saturated (section 3.7), or the baud rates differ. **Fix:** Use 460 800 on both sides, or at 115 200 raise `IMU_ODOM_PUB_DIV` to 4 or more. Check with `ros2 topic bw`.

Error 16: `/self_check` or `/heading_hold_cmd` never appears

Cause: The subscription pool is full (section 3.4); the two optional entities are created last and fail quietly. **Fix:** Raise `RMW_UXRCE_MAX_SUBSCRIPTIONS` in `custom.meta`, rebuild the micro-ROS library, and re-flash.

6.2 Sensors and Actuators

Error 7: servos move together (ghosting)

Original cause: Hardware timer sharing on the ESP32-S3. **Original fix:** Removing one servo and attaching the dump servo only while in use. The current firmware has two servos attached permanently with all timers allocated. Whether the problem returns has not been re-tested.†If it does, restore the attach/detach approach for the dump servo.

Error 8: yaw drifts and the robot veers

Cause: Integrating the gyro adds up its zero-rate offset; separately, motors differ. **Fix:** Boot-time bias calibration, the stand-still bias update, a noise gate after bias subtraction, and heading hold (sections 4.4 and 4.6). Yaw still drifts slowly because there is no magnetometer; correct it from the lidar.

Error 9: nonsense IMU values

Cause: `Wire.read()≪8 | Wire.read()` has unspecified evaluation order, so bytes may swap. **Fix:** Read each byte into its own variable first.

Error 10: `/imu_data` silent or wrong timestamps

Cause (suspected):†A `frame_id` string with zero capacity cannot be serialised, and `RCSOFTCHECK` hides the failure; the timestamp was never set. **Fix:** Give `frame_id` a static buffer with correct size and capacity, and stamp from `rmw_uros_epoch_nanos()`.

Error 11: stepper stutters, IMU timing jitters

Cause: `tcs.getRawData()` waits about one integration time (reasoned from the library, not measured), and a FreeRTOS task with a mutex only moved the stall. **Fix:** Read the sensor registers directly in `loop()` (chapter 4).

6.3 Dashboard and Network

Error 12: encoder boxes swapped left/right

Cause: The firmware order is FL, RL, FR, RR. **Fix:** Map indices 0–3 to FL, RL, FR, RR (the current dashboard does).

Error 13: dashboard unreachable from a phone

Cause: WSL 2 uses NAT, so ports 8000 and 9090 are not visible on the Wi-Fi. **Fix:** Mirrored networking: create `C:\Users\<you>\.wslconfig` containing `[wsl2]` and `networkingMode=mirrored`, run `wsl -shutdown`, allow ports 8000 and 9090 in Windows Defender Firewall, set the Wi-Fi network to Private, and open `http://<laptop-wifi-ip>:8000/dashboard.html`. A Hyper-V firewall rule may also be needed; the exact command from the earlier draft was given from memory and must be checked against Microsoft's WSL documentation.† Campus or hostel Wi-Fi often isolates clients; a phone hotspot avoids that. A `GET /sw.js` 404 in the web-server log is harmless.

Error 15: telemetry panels stay empty but the drive buttons work

Cause: QoS mismatch (section 3.8). **Fix:** `ros2 topic info /imu_data -v`; make the host subscriber best-effort. As a last resort, change the three sensor publishers to reliable and accept the stall risk on a slow link.

Error 17: wrong self-check labels on the dashboard

Cause: Dashboard and firmware disagree on the item list (table 5.2, #1). **Fix:** Remove "Pan servo" from the dashboard list.

Error 18: dashboard blank on a device without internet

Cause: `roslib` and `eventemitter2` are fetched from a CDN. **Fix:** Download both files into the served folder and change the two `<script>` paths.

Error 19: "extrapolation" or "message too old" in Nav2 or RViz

Cause: The ESP32 clock synchronisation failed, so stamps are in ESP32 time (section 3.5). **Fix:** Restart the agent and let the ESP32 reconnect; check that the stamp on `ros2 topic echo /imu_data -field header` is close to `date +%s`.

Appendix A

Items Not Yet Verified

Each item carries a dagger † in the text. Test them on the robot and delete the mark when confirmed.

1. `pio run -t clean_microros` forces a rebuild of the micro-ROS library after editing `custom.meta`.
2. Host subscribers (Nav2, `robot_localization`, rosbridge) must use best-effort QoS to receive the ESP32's sensor topics, and whether rosbridge adapts automatically.
3. The coast-versus-brake PWM scheme as the cause of the direction-dependent gain pattern.
4. The EKF consistency gate has no effect while the yaw innovation is forced to zero (calculation only).
5. Whether servo ghosting (Error 7) returns with two servos permanently attached.
6. The suspected cause of Error 10 (zero-capacity `frame_id`).
7. The Hyper-V firewall command for WSL mirrored networking.
8. All payload sizes in table 3.6, the 1.7 s disconnect-detection time, and the loop blocking durations are calculated from source constants, not measured.

Appendix B

Glossary

Agent The program on the host that forwards micro-ROS messages to ordinary ROS 2 (section 2.7).

Client The micro-ROS library running on the ESP32.

DDS The messaging system underneath ROS 2. **DDS-XRCE** is a compact version for microcontrollers.

Entity Anything micro-ROS creates: a node, a publisher or a subscription.

Executor The part of `rclc` that checks for incoming messages and calls your callbacks.

Callback A function that runs when a message arrives.

Node, topic A node is a program in the ROS 2 graph; a topic is a named channel of messages of one type.

Publisher, subscription The sending and receiving ends of a topic.

QoS Quality of Service: how a topic is delivered, mainly *reliable* (retried) or *best-effort* (may drop).

Baud rate Serial link speed in bits per second; both ends must use the same.

rosbridge A server that lets a web page use ROS 2 topics over a WebSocket.

`twist_mux` A node that picks one velocity command source by priority.

Odometry, EKF Estimate of where the robot is from its own sensors; an Extended Kalman Filter is the method used to combine them.

PlatformIO The build and flash tool for the firmware.

WSL 2, `usbipd` Linux inside Windows, and the tool that hands a USB device to it.

Bibliography

[1] K. Belsare *et al.*, "Micro-ROS," in *Robot Operating System (ROS): The Complete Reference, Vol. 7*, Springer, 2023.

[2] Object Management Group, *DDS For eXtremely Resource Constrained Environments (DDS-XRCE)*, v1.0, 2020.

[3] S. Macenski *et al.*, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, 2022.